Claude Code ワークフロー

AI 駆動開発の実践

伊藤 恒平

2026

このスライドについて / Purpose

概要

StampFly Ecosystem のスライド・ファームウェア・ツールはすべて **Claude Code**（Anthropic の CLI エージェント）と人間の協働で開発された

- 817 コミット / 70 日間 — アクティブ日平均 20 コミット
- fix 329 件、feat 225 件、docs 140 件
- CLAUDE.md の継続的改善で品質を担保

ゴール

実プロジェクトで得た Claude Code のワークフローと Tips を共有する

Claude Code とは / What is Claude Code?

概要

Anthropic 公式の CLI エージェント。ターミナルから Claude を呼び出し、コード読み書き・ビルド・テスト・Git 操作を自律的に実行する

機能	説明
CLAUDE.md	プロジェクト固有の指示書（常に読み込まれる）
Memory	セッション間で記憶を永続化（4 タイプ）
Plan Mode	Shift+Tab で読み取り専用の設計モード
Subagent	並列タスク・重い処理を子プロセスに委任
Hooks	ツール実行前後にシェルコマンドを自動実行
Custom Skills	`/commit` 等のカスタムワークフロー
MCP Server	外部サービス連携（DB、draw.io 等）

公式: `https://docs.anthropic.com/en/docs/claude-code`

CLAUDE.md — プロジェクトの指示書

階層構造（すべて同時に読み込まれる）

`~/.claude/CLAUDE.md`	グローバル設定（全プロジェクト共通）
`<repo>/CLAUDE.md`	プロジェクト設定（Git 管理）
`<subdir>/CLAUDE.md`	ディレクトリ固有の設定
`.claude/rules/*.md`	パス限定ルール（paths フロントマター）

書くべき内容

- ビルド・テストコマンド / コーディング規約 / コミット形式
- **やってはいけないこと**（アンチパターン）← 最重要
- プロジェクト構造の概要

CLAUDE.md の実例 / Real Example

```
Slide Rules **CRITICAL: .tex 変更後は必ず PDF リビルド+レビュー**
Slide Review Checklist | ID | チェック内容 | |----|--------------------------------| | L1 | フッターにはみ出していないか | | T4 | 矢印は水平・垂直のみ | | T8 | resizeboxwh 両方指定は禁止 |
```

チェックリストは実際の失敗事例から段階的に構築された（後述）

Memory — セッション間の記憶

4 つのメモリタイプ

Type	用途
user	ユーザーの専門性・好み
feedback	修正指示・NG パターン
project	進行中の作業・期限
reference	外部リソースへのリンク

feedback メモリの実例

速度優先は手戻りを招き工数増大。
チェックリストは 1 回もスキップしない。

サブエージェントの判定を鵜呑みにしない。

一度保存すれば、**全ての将来のセッション**で Claude がこのルールに従う

Tips: コードや Git 履歴から分かる情報は保存しない（二重管理の回避）

Plan Mode — 実装前に設計する

Plan Mode とは

Shift+Tab でトグル。コード編集を**できない**モード。
調査・設計に集中させ、方針が固まってから実装に移る

有効な場面

- 複数ファイルにまたがる変更
- アーキテクチャの判断が必要
- 失敗コストが高い作業

典型的なフロー

1. Plan Mode で調査・設計
2. ユーザーが計画を承認
3. 実装 → テスト → コミット
4. 問題発見 → 計画を更新

Tips:「思考」と「実行」を分離することで手戻りを大幅に削減できる

Subagent — 並列処理とコンテキスト保護

タイプ	用途	説明
general-purpose	汎用タスク	全ツール利用可。複雑な作業を委任
Explore	コード検索	読み取り専用。高速なコードベース探索
Plan	設計	読み取り専用。アーキテクチャ設計

このプロジェクトでの活用例

- **画像レビュー** — PDF を画像化して目視確認。メインに画像を持ち込まない
- **並列リサーチ** — 複数サブエージェントを同時起動して情報収集
- **バックグラウンド実行** — ビルドを裏で実行し完了通知を受け取る

サブエージェントの「問題なし」を鵜呑みにしない — メイン側で検証すること

Hooks & Skills — 自動化の仕組み

Hooks（自動実行）

Event	用途
PreToolUse	危険操作のブロック
PostToolUse	自動 lint / format
Notification	Slack 通知転送
Stop	応答完了後の自動テスト

`settings.json` で設定。

prompt-based / agent-based も可能

Custom Skills

`/commit` — ガイドライン準拠の Git コミットを自動作成

`/simplify` — コード品質チェック

`/batch` — 並列ワークツリーで大規模変更を一括処理

`.claude/skills/` に SKILL.md で定義。

`$ARGUMENTS` で引数を受け取れる

Tips: 繰り返す作業は Skill に、条件付き自動化は Hook にまとめる

コンテキスト管理 / Context Management

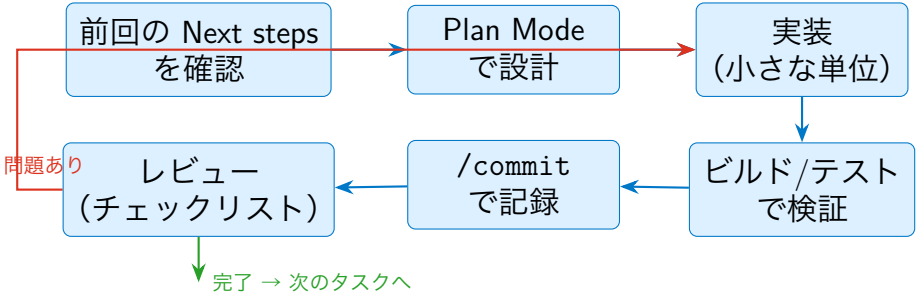
コンテキストウィンドウは最重要リソース

Claude が一度に扱える情報量には上限。無駄に消費すると性能劣化し指示を忘れる

手法	説明
`/compact`	会話履歴を要約して圧縮（~95% で自動発動）
Subagent 委任	大量の検索結果をメインに持ち込まない
Memory 永続化	繰り返し伝える情報はメモリに保存
CLAUDE.md 簡潔化	毎回読み込まれるので 200 行以内に
画像はサブエージェント	PDF/画像の Read はコンテキスト消費大

教訓: PDF 全ページ確認をメインで行い rate limit 抵触 → サブエージェント限定ルールを制定

実践ワークフロー / Workflow in Practice



Git コミット戦略 / Commit Strategy

Conventional Commits + Next steps

```
Changes: - TikZ 座標を調整しノード貫通を解消- adjustbox で均等スケーリングに変更
Next steps: - sf build vehicle でビルド確認- P34-P40 の同様の問題を確認
```

自動コミットのトリガー:

- ファイル変更後
- テスト/ビルド成功後
- 「ここまで」「休憩」等の発言

Next steps の効果:

- セッション再開時の起点になる
- 817 コミット中 318 件に記載
- 具体的なコマンド・検証項目を含む

失敗から学ぶ / Lessons from Failures

事例: P33 espnow_dataflow — 7 回の fix コミット

1 枚のスライドの TikZ 図修正にリビルドサイクルが 12 回以上発生

何が起きたか:

1. `resizebox{w}{h}` でフォント縦伸び
2. 矢印方向が上段/下段で不統一
3. 場当たり的に Beamer 側で対処
4. チェックリスト未適用で見落とし

得られた教訓:

1. 根本原因を最初に特定する
2. チェックリストは 1 回も省略しない
3. サブエージェントの判定を鵜呑みにしない
4. CLAUDE.md に失敗事例を記録 → 再発防止

対策

feedback メモリに保存 → 全セッションで Claude がこのルールに従うようになった

反復改善のパターン / Iteration Patterns

領域	Fix	内容
installer	18	WSL2、Windows パス、Python 検出
シミュレータカメラ	13	VPython カメラ追従（revert 2 回含む）
P33 TikZ 図	7	ルーティング、アスペクト比、ノード貫通
L10/L11 ヘッダー	5	framesubtitle 削除後のレイアウト崩れ
エコシステム図	5	矢印ラベル重なり → 直角ルーティング
コントローラメニュー	4	フリッカー、残像、スクロール

CLAUDE.md の進化 / Evolution of CLAUDE.md

Jan: ビルドコマンド、コーディング規約、sf CLI 使用方針

↓

Jan: コミットガイドライン + Next steps ルール追加

↓

Feb: スライドレビューチェックリスト導入（L1–L6, T1–T9）

↓

Feb: サブエージェントによる画像レビュー方式に切替

↓

Mar: サブエージェント限定ルール強制化（rate limit 対策）

↓

Mar: TikZ チェックリスト大幅拡充（T4 直角ルーティング等）

↓

Mar: feedback メモリで品質優先の原則を永続化

統計で見るプロジェクト / Project by Numbers

指標	値
総コミット数	817
開発期間	70 日間（2026/1/5 – 3/15）
アクティブ日の平均	20.4 コミット/日
最多の日	63 コミット（1/12）
fix 比率	40.3%（329/817）
feat 比率	27.5%（225/817）
最長 fix チェイン	18 回（installer）
revert 回数	10 回（1.2%）
Next steps 記載率	38.9%（318/817）
CLAUDE.md 改訂回数	17 回

Top 10 Tips

1. **CLAUDE.md は基盤** — ビルドコマンド、規約、禁止事項を書く
2. **ガードレール** > **指示** —「やるな」の方が「やれ」より効果的
3. **Plan** → **Implement** → **Test** — 考えてから書く
4. **小さくコミット** — 1 機能 = 1 コミット、Next steps を含める
5. **チェックリストは省略しない** — 速度優先は手戻りで工数増大
6. **feedback メモリで学習** — 同じミスを二度と繰り返さない
7. **サブエージェントで並列化** — コンテキストを守りつつ効率化
8. **画像レビューはサブエージェントに** — rate limit 対策
9. **CLAUDE.md を育てる** — 失敗するたびにルールを追加
10. **コンテキストは最重要リソース** — `/compact` と Memory を活用

参考資料 / References

リソース	URL
公式ドキュメント	https://docs.anthropic.com/en/docs/claude-code
ベストプラクティス	https://docs.anthropic.com/en/docs/claude-code/best-practices
Claude Code GitHub	https://github.com/anthropics/claude-code

Claude Code を始めるには

```
npm install -g @anthropic-ai/claude-code
cd your-project && claude
```